Functional Requirements
Tower Defence Game

Jonas Leipner

November 2, 2026

1 Introduction

This document translates the project's initial user stories into functional requirements. The requirements are organised around three personas:

- **Player:** the person who plays the tower defence game, places towers, and responds to incoming enemies.
- **Researcher:** the person who analyses player behaviour and uses telemetry to investigate player strategies.
- **System Architecture:** the architectural responsibilities concerned with extracting game state and executing game logic dynamically.

The project is being developed iteratively using an Agile/Scrum-inspired approach. The requirements should therefore be treated as a prioritised product backlog: Must Have items define the initial product increment, while Should Have and Could Have items can be scheduled into later iterations if they remain compatible with the project's time and research scope.

2 MoSCoW Prioritisation

The MoSCoW method is used to control scope:

- **Must Have:** Essential for the minimum viable product or for testing the central architectural experiment.
- **Should Have:** Important and valuable, but the system can still function without it in the first product increment.
- **Could Have:** Desirable enhancements that are implemented only if sufficient time and resources remain.
- **Won't Have:** Explicitly excluded from the current project scope, although it may be reconsidered in future work.

3 Must Have Requirements

3.1 Player Persona

- **FR-M1 — Tower placement:** The system shall allow the player to select towers and place a maximum of five towers on the game grid. The placed towers shall be usable to defend the home base from incoming monsters.

This requirement defines the basic game loop. Without tower placement and a limit on the number of towers, there is no meaningful defensive gameplay.

User story: As a player, I want to select towers and place a maximum of five on the grid so that I can defend my home base from incoming monsters.

Acceptance criteria:

- Given that fewer than five towers have been placed, when the player selects a valid tower position, then the selected tower is placed on the grid.
- Given that five towers have been placed, when the player attempts to place another tower, then the placement is rejected and the number of placed towers remains five.
- Given that the player selects an invalid or occupied position, when they attempt to place a tower, then the placement is rejected and no existing tower is moved or removed.

3.2 System Architecture Persona

- **FR-M2 — Dynamic combat logic:** The system shall extract the current combat state from the Flame game engine and pass that state to a `dart_eval` sandbox. The sandbox shall be capable of executing game logic dynamically without requiring an application restart or a new ahead-of-time compilation.

This requirement is the central architectural experiment of the project. The game may be playable without dynamic evaluation, but the project objective depends on demonstrating this Flame-to-`dart_eval` boundary.

User story: As the system architecture, I want to extract the current combat state from Flame and pass it to a `dart_eval` sandbox so that game logic can be executed dynamically without needing an app restart or a new AOT compilation.

Acceptance criteria:

- Given an active Flame combat session, when the dynamic evaluation step is triggered, then the current combat state is serialised into the agreed state representation and passed to the `dart_eval` sandbox.
- Given a valid combat state and evaluable game-logic code, when the sandbox executes, then it returns a result that can be applied to the active game session.
- Given that the game is already running, when new evaluable logic is supplied, then the logic can be executed without restarting the application or producing a new AOT compilation.

4 Should Have Requirements

4.1 Player Persona

- **FR-S1 — Enemy scoring:** The system should assign different point values to different enemy types. The player should be able to use the resulting score to measure success as game waves become progressively more difficult.

Different enemy values improve feedback, enjoyment, and progression. However, the core defensive game loop can function without a fully developed scoring system.

User story: As a player, I want enemies to be worth different point values so that I can track my score and measure my success as the waves get progressively harder.

Acceptance criteria:

- Given two enemy types with different configured values, when the player defeats them, then the score increases by the corresponding value for each enemy.
- Given a completed wave, when the player views the game status, then the current score is displayed.
- Given that waves become progressively harder, when the player defeats enemies in later waves, then the score continues to reflect the configured values of those enemies.

4.2 Researcher Persona

- **FR-S2 — Tower telemetry:** The system should capture telemetry data describing which towers are placed most frequently. The data should support analysis of player strategies and provide a possible foundation for a future dynamic recommender system.

Telemetry supports the research goals but is not required for the game to be playable. The first implementation should therefore remain lightweight, such as a local structure recording tower-placement events.

User story: As a researcher, I want to capture telemetry data on which towers are placed most frequently so that I can analyse optimal player strategies and eventually power a dynamic recommender system.

Acceptance criteria:

- Given that a player places a tower, when the placement succeeds, then a local telemetry event records the tower type and placement event.
- Given that multiple placements have been recorded, when telemetry is queried, then the system can report the placement count for each tower type.
- Given that the game is played without a network connection, when tower placements occur, then the telemetry is retained locally rather than being silently discarded.

5 Could Have Requirements

5.1 Player Persona

- **FR-C1 — Tower merging:** The system could allow the player to merge two towers in an incubator to create a unique tower. This feature could allow players to discover new defensive options and achieve a higher score.

Tower merging could provide a distinctive gameplay mechanic, but it introduces additional algorithmic, interface, balancing, and testing complexity. It is therefore outside the immediate minimum viable product.

User story: As a player, I want to merge two towers in the incubator to create a unique tower so that I can discover new defences and achieve a higher score.

Acceptance criteria:

- Given two compatible towers in the incubator, when the player confirms the merge, then the two input towers are removed and the defined unique tower is created.
- Given two incompatible towers, when the player attempts to merge them, then the merge is rejected and both input towers remain unchanged.
- Given a successful merge, when the resulting tower is displayed, then the player can identify its type and use it as a defensive tower.

5.2 System Architecture Persona

- **FR-C2 — Local outcome logging:** The system could log player choices and victory metrics locally. The logged data could later be formatted and supplied to a dynamic recommender system.

This requirement is a possible future research direction rather than a fixed part of the initial project scope. It can be considered after the core architecture and gameplay loop have been demonstrated.

User story: As the system architecture, I want to log player choices and victory metrics locally so that the data can later be formatted and fed into a dynamic recommender system.

Acceptance criteria:

- Given that the player makes a tracked choice, when the choice is completed, then a local log entry records the choice and its timestamp.
- Given that a game ends, when the player wins or loses, then the local log records the outcome and the relevant victory metrics.
- Given a collection of local log entries, when the data is exported, then it is represented in a consistent format that can be processed by a future recommender system.

6 Won't Have Requirements

No requirements have currently been assigned to the Won't Have category. Requirements placed in this category would be explicitly excluded from the current project scope and would not be scheduled into the planned increments.

7 Prioritised Product Backlog

Table 1 provides a compact backlog view for planning iterations or Scrum sprints.

Table 1: Prioritised functional requirements

ID	Priority	Requirement
FR-M1	Must Have	Place a maximum of five towers on the grid to defend the home base.
FR-M2	Must Have	Pass combat state from Flame to a `dart_eval` sandbox for dynamic game logic.
FR-S1	Should Have	Assign different point values to enemy types and use them for player scoring.
FR-S2	Should Have	Capture telemetry about frequently placed towers.
FR-C1	Could Have	Merge two towers to create a unique tower.
FR-C2	Could Have	Log player choices and victory metrics locally.

8 Agile Use

At the start of each iteration, the team can select a small number of requirements from the backlog and break them into implementation tasks, tests, and acceptance criteria. A requirement should be considered complete only when its behaviour has been implemented, tested, and demonstrated. The backlog can be updated after each review if project findings or supervisor feedback change the priority of a requirement.